

```
mvnw spring-boot:run
mvnw clean install
npx create-react-app nome
npm install
npm start
```

```
npx run dev
```

```
npm install react-router-dom
```

web, data, lombok, driver banco, dev tools, validation.

Para resolver problemas de CORS, caso não exista uma classe de configuração específica, adicione a anotação `@CrossOrigin` diretamente sobre a classe de controle.

```
import "primereact/resources/themes/lara-light-indigo/theme.css";
import "primereact/resources/primereact.min.css";
import "primeicons/primeicons.css";
```

```
salvar({memorySize:objeto.memory_size})
```

```
@Query("from Pessoa where email=:email")
public Page<Pessoa> buscarEmail(@Param("email") String email, Pageable pageable);
```

```
package com.leilao.backend.controller;
```

```
import java.util.List;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
import com.leilao.backend.model.Profile;
import com.leilao.backend.service.ProfileService;
```

```
import jakarta.websocket.server.PathParam;
```

```
@RestController
@RequestMapping("/api/profile")
public class ProfileController {
```

```
    @Autowired
    private ProfileService profileService;
```

```
@PostMapping
public Profile create(@RequestBody Profile profile) {
    return profileService.create(profile);
}
```

```
@PutMapping
public Profile update(@RequestBody Profile profile) {
    return profileService.create(profile);
}
```

```
@GetMapping
public List<Profile> listAll() {
    return profileService.listAll();
}
```

```
// localhost:8080/api/profile/10
@DeleteMapping("/{id}")
public void delete(@PathVariable("id") Long id) {
    profileService.delete(id);
}
```

```
// localhost:8080/api/profile?name=jose&age=2
@GetMapping("/find")
public String find(@PathParam("name") String name,
    @PathParam("age") Integer age) {
    System.out.println(name + " " + age);
    return name + " " + age;
}
}
```

```
package com.leilao.backend.service;
```

```
import java.util.List;
import java.util.NoSuchElementException;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
```

```
import com.leilao.backend.model.Auction;
import com.leilao.backend.model.Person;
import com.leilao.backend.repository.AuctionRepository;
import com.leilao.backend.security.AuthPersonProvider;
```

```
@Service
public class AuctionService {
```

```
    @Autowired
    private AuctionRepository auctionRepository;
```

```

    @Autowired
    private AuthPersonProvider authPersonProvider;

    public Auction create(Auction auction) {
        auction.setPerson(authPersonProvider.getAuthenticatedUser());
        return auctionRepository.save(auction);
    }

    public Auction update(Auction auction) {
        Auction auctionSaved = auctionRepository.findById(auction.getId())
            .orElseThrow(() -> new NoSuchElementException("Leilão não encontrado"));
        auctionSaved.setTitle(auction.getTitle());
        auctionSaved.setDescription(auction.getDescription());
        auctionSaved.setStartDateTime(auction.getStartDateTime());
        auctionSaved.setEndDateTime(auction.getEndDateTime());
        auctionSaved.setStatus(auction.getStatus());
        auctionSaved.setObservation(auction.getObservation());
        auctionSaved.setIncrementValue(auction.getIncrementValue());
        return auctionRepository.save(auctionSaved);
    }

    public void delete(Long id) {
        Auction auctionSaved = auctionRepository.findById(id)
            .orElseThrow(() -> new NoSuchElementException("Leilão não encontrado"));
        auctionRepository.delete(auctionSaved);
    }

    public List<Auction> listAll() {
        Person authenticatedUser = authPersonProvider.getAuthenticatedUser();
        return auctionRepository.findByPerson(authenticatedUser);
    }

    public List<Auction> listAllPublic() {
        return auctionRepository.findAll();
    }

    public Auction findById(Long id) {
        Person authenticatedUser = authPersonProvider.getAuthenticatedUser();
        Auction auction = auctionRepository.findByIdAndPerson(id, authenticatedUser);
        if (auction == null) {
            throw new NoSuchElementException("Leilão não encontrado ou não pertence ao usuário autenticado");
        }
        return auction;
    }
}

package com.leilao.backend.service;

```

```

import org.springframework.stereotype.Service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.thymeleaf.TemplateEngine;
import org.thymeleaf.context.Context;

import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;

@Service
public class EmailService {

    @Autowired
    private JavaMailSender mailSender;

    @Autowired
    private TemplateEngine templateEngine;

    /**
     * Envia um email simples com o destinatário, assunto e texto fornecidos.
     *
     * @param to o endereço de email do destinatário
     * @param subject o assunto do email
     * @param text o conteúdo do email
     */
    public void sendSimpleEmail(String to, String subject, String text) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setTo(to);
        message.setSubject(subject);
        message.setText(text);
        mailSender.send(message);
    }

    /**
     * Envia um email usando um template com o destinatário, assunto, variáveis de
     * email e nome do arquivo de template fornecidos.
     *
     * @param to o endereço de email do destinatário
     * @param subject o assunto do email
     * @param emailVariables as variáveis de email a serem usadas no template -
     * org.thymeleaf.context.Context
     * @param templateFileName o nome do arquivo html de template
     * @throws MessagingException se ocorrer um erro ao enviar o email
     */
    public void sendTemplateEmail(String to, String subject, Context emailVariables, String
templateFileName)
        throws MessagingException {

```

```

        String process = templateEngine.process(templateFileName, emailVariables);

        MimeMessage message = mailSender.createMimeMessage();
        MimeMessageHelper helper = new MimeMessageHelper(message, true);

        helper.setTo(to);
        helper.setSubject(subject);
        helper.setText(process, true);

        mailSender.send(message);
    }
}

```

@startuml

```

enum StatusLeilao {
    ABERTO
    ENCERRADO
    CANCELADO
    EM_ANALISE
}

```

```

enum TipoPerfil {
    ADMIN
    COMPRADOR
    VENDEDOR
}

```

```

class Pessoa {
    - nome: String
    - email: String
    - senha: String
    - codigoValidacao: String
    - validadeCodigoValidacao: Date
    - ativo: Boolean
    - fotoPerfil: Lob
}

```

```

class Perfil {
    - tipo: TipoPerfil
}

```

```

class PessoaPerfil {
}

```

```

class Categoria {
    - nome: String
    - observacao: String
}

```

```

class Leilao {
    - titulo: String
    - descricao: String
    - descricaoDetalhada: String
    - dataHoraInicio: DateTime
    - dataHoraFim: DateTime
    - status: StatusLeilao
    - observacao: String
    - valorIncremento: Float
    - lanceMinimo: Float
}

```

```

class Imagem {
    - dataHoraCadastro: DateTime
    - nomeImagem: String
}

```

```

class Lance {
    - valorLance: Float
    - dataHora: DateTime
}

```

```

class Feedback {
    - comentario: String
    - nota: Integer
    - dataHora: DateTime
}

```

```

class Pagamento {
    - valor: Float
    - dataHora: DateTime
    - status: String
}

```

'==== Relacionamentos ====

Pessoa "0..\*" <-> "1..\*" Perfil  
(Pessoa, Perfil) .. PessoaPerfil : classe associativa

Pessoa "1" --> "0..\*" Categoria : cria  
Pessoa "1" --> "0..\*" Leilao : publica  
Pessoa "1" --> "0..\*" Lance : realiza  
Pessoa "1" --> "0..\*" Feedback : escreve  
Feedback "1" --> "1" Pessoa : destinatario

Leilao "1" --> "1" Categoria  
Leilao "1" <--> "0..\*" Imagem  
Leilao "1" --> "0..\*" Lance  
Leilao "1" --> "0..1" Pagamento

@enduml

```
package com.leilao.backend.model;
```

```
import java.time.LocalDateTime;  
import java.util.Collection;  
import java.util.Date;  
import java.util.List;  
import java.util.stream.Collectors;
```

```
import org.hibernate.validator.constraints.UniqueElements;  
import org.springframework.security.core.GrantedAuthority;  
import org.springframework.security.core.authority.SimpleGrantedAuthority;  
import org.springframework.security.core.userdetails.UserDetails;  
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;  
import org.springframework.security.crypto.password.PasswordEncoder;
```

```
import com.fasterxml.jackson.annotation.JsonIgnore;  
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;  
import com.fasterxml.jackson.annotation.JsonIgnoreType;  
import com.fasterxml.jackson.annotation.JsonProperty;
```

```
import jakarta.persistence.CascadeType;  
import jakarta.persistence.Column;  
import jakarta.persistence.Entity;  
import jakarta.persistence.FetchType;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;  
import jakarta.persistence.OneToOne;  
import jakarta.persistence.Table;  
import jakarta.persistence.Temporal;  
import jakarta.persistence.TemporalType;  
import jakarta.persistence.Transient;  
import jakarta.validation.constraints.Email;  
import jakarta.validation.constraints.Min;  
import jakarta.validation.constraints.NotBlank;  
import lombok.AccessLevel;  
import lombok.Data;  
import lombok.Setter;
```

@Entity

@Data

@Table(name = "person")

@JsonIgnoreProperties({ "authorities", "cpf" })

public class Person implements UserDetails {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

```

private Long id;

@NotBlank(message = "{name.required}")
private String name;

>Email(message = "{name.invalid}")
@UniqueElements(message = "E-mail já cadastrado")
private String email;

// @CPF
private String cpf;

@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
private String password;

@Transient
private static final PasswordEncoder passwordEncoder = new BCryptPasswordEncoder();

public void setPassword(String password) {
    this.password = passwordEncoder.encode(password);
}

@JsonIgnore
@Column(name = "validation_code")
private Integer validationCode;

@Temporal(TemporalType.TIMESTAMP)
private Date validationCodeValidity;

@OneToMany(mappedBy = "person", orphanRemoval = true, cascade = CascadeType.ALL,
fetch = FetchType.EAGER)
@Setter(value = AccessLevel.NONE)
private List<PersonProfile> personProfile;

public void setPersonProfile(List<PersonProfile> lpp) {
    for (PersonProfile p : lpp) {
        p.setPerson(this);
    }
    personProfile = lpp;
}

@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    return personProfile.stream()
        .map(userRole -> new SimpleGrantedAuthority(userRole.getProfile().getName()))
        .collect(Collectors.toList());
}

@Override
public String getUsername() {

```

```
        return email;
    }
}
```

```
package com.leilao.backend.repository;
```

```
import java.util.List;
import java.util.Optional;
```

```
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
```

```
import com.leilao.backend.model.Pessoa;
```

```
public interface PessoaRepository extends JpaRepository<Pessoa, Long> {
```

```
    @Query("from Pessoa where email=:email")
    public Page<Pessoa> buscarEmail(@Param("email") String email, Pageable pageable);

    public Optional<Pessoa> findByEmail(String email);
}
```

```
package com.leilao.backend.security;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;
```

```
import com.leilao.backend.model.Pessoa;
import com.leilao.backend.repository.PessoaRepository;
```

```
import java.util.NoSuchElementException;
```

```
@Component
```

```
public class AuthPessoaProvider {
```

```
    @Autowired
    private PessoaRepository userRepository;
```

```
    public Pessoa getUsuarioAutenticado() {
        Object principal = SecurityContextHolder.getContext().getAuthentication().getPrincipal();
        String username;
```

```
        if (principal instanceof UserDetails) {
            username = ((UserDetails) principal).getUsername();
            System.out.println(username);
        }
    }
}
```

```

    } else {
        username = principal.toString();
    }

    return userRepository.findByEmail(username)
        .orElseThrow(() -> new NoSuchElementException("Usuário autenticado não
encontrado"));
    }
}

package com.leilao.backend.security;

import java.util.List;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfigur
ation;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class ConfiguracaoSeguranca {

    private final JwtFiltroAutenticacao jwtRequestFilter;

    public ConfiguracaoSeguranca(JwtFiltroAutenticacao jwtRequestFilter) {
        this.jwtRequestFilter = jwtRequestFilter;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

```

```

@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration
authenticationConfiguration) throws Exception {
    return authenticationConfiguration.getAuthenticationManager();
}

```

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http.csrf(csrf -> csrf.disable())
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        .authorizeHttpRequests(authorize -> authorize
            .requestMatchers("/autenticacao/**").permitAll()
            .requestMatchers("/categoria/**").permitAll()
            .requestMatchers("/leilao/public").permitAll()
            /* .requestMatchers("/api/pessoa/**").hasRole("ADMIN") */
            .anyRequest().authenticated()
        )
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        )
        .addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

```

```

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();
    configuration.setAllowedOrigins(List.of("http://localhost:3000"));
    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    configuration.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    configuration.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);

    return source;
}
}

```

```

package com.leilao.backend.security;

```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

```

```

import com.leilao.backend.service.PessoaService;

```

```

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

@Component
public class JwtFiltroAutenticacao extends OncePerRequestFilter {

    @Autowired
    private JwtService jwtService;

    @Autowired
    private PessoaService pessoaService;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
    FilterChain chain)
        throws ServletException, IOException {
        String authorizationHeader = request.getHeader("Authorization");
        String token = null;
        String username = null;

        if (authorizationHeader != null && authorizationHeader.startsWith("Bearer ")) {
            token = authorizationHeader.substring(7);
            username = jwtService.extractUsername(token);
        }

        if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            var userDetails = pessoaService.loadUserByUsername(username);
            if (jwtService.validateToken(token, userDetails.getUsername())) {
                var authentication = new UsernamePasswordAuthenticationToken(
                    userDetails, null, userDetails.getAuthorities()
                );
                authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authentication);
            }
        }
        chain.doFilter(request, response);
    }
}

package com.leilao.backend.security;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import jakarta.annotation.PostConstruct;

```

```

import io.jsonwebtoken.Claims;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.security.Key;
import java.util.Date;
import java.util.function.Function;

@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secretKey;

    @Value("${jwt.expiration}")
    private Long expiration;

    private Key key;

    @PostConstruct
    public void init(){
        key = Keys.hmacShaKeyFor(secretKey.getBytes());
    }

    public String generateToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + expiration))
            .signWith(key, SignatureAlgorithm.HS256)
            .compact();
    }

    public Boolean validateToken(String token, String username) {
        String extractedUsername = extractUsername(token);
        return (username.equals(extractedUsername) && !isTokenExpired(token));
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(key)

```

```

        .build()
        .parseClaimsJws(token)
        .getBody();
    }

    private Boolean isTokenExpired(String token) {
        return extractClaim(token, Claims::getExpiration).before(new Date());
    }
}

package com.leilao.backend.exception;

import java.util.List;
import java.util.stream.Collectors;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;

import com.leilao.backend.dto.RespostaErro;

@RestControllerAdvice
public class ExcecaoGlobal {

    @ExceptionHandler(NaoEncontradoExcecao.class)
    public ResponseEntity<RespostaErro> naoEncontrado(NaoEncontradoExcecao ex,
    WebRequest request) {
        RespostaErro respostaErro = new RespostaErro(HttpStatus.NOT_FOUND.value(), "Não
    Encontrado",
            ex.getMessage(),
            request.getDescription(false), null);
        return new ResponseEntity<>(respostaErro, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(NegocioExcecao.class)
    public ResponseEntity<RespostaErro> negocio(NegocioExcecao ex, WebRequest request) {
        RespostaErro respostaErro = new
    RespostaErro(HttpStatus.UNPROCESSABLE_ENTITY.value(), "Erro de Negócio",
            ex.getMessage(),
            request.getDescription(false), null);
        return new ResponseEntity<>(respostaErro, HttpStatus.UNPROCESSABLE_ENTITY);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<RespostaErro> validacao(MethodArgumentNotValidException ex,
    WebRequest request) {

```

```

        List<String> erros = ex.getBindingResult().getFieldErrors().stream()
            .map(err -> err.getField() + ": " + err.getDefaultMessage()).collect(Collectors.toList());

        RespostaErro respostaErro = new RespostaErro(HttpStatus.BAD_REQUEST.value(), "Erro
de Validação",
            "Campos Inválidos",
            request.getDescription(false), erros);

        return new ResponseEntity<>(respostaErro, HttpStatus.BAD_REQUEST);
    }

    // método global
    @ExceptionHandler(Exception.class)
    public ResponseEntity<RespostaErro> global(Exception ex, WebRequest request) {
        RespostaErro respostaErro = new
RespostaErro(HttpStatus.INTERNAL_SERVER_ERROR.value(), "Erro Interno",
            ex.getMessage(),
            request.getDescription(false), null);
        return new ResponseEntity<>(respostaErro, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

```
package com.leilao.backend.exception;
```

```

public class NaoEncontradoExcecao extends RuntimeException {

    public NaoEncontradoExcecao(String mensagem) {
        super(mensagem);
    }

}

```

```
package com.leilao.backend.exception;
```

```

public class NegocioExcecao extends RuntimeException {

    public NegocioExcecao(String mensagem) {
        super(mensagem);
    }

}

```

```
package com.leilao.backend.model;
```

```
import org.hibernate.validator.constraints.br.CPF;
```

```

import jakarta.annotation.Generated;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;

```

```
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.Data;
```

```
@Entity
@Data
@Table(name = "perfil")
public class Perfil {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    @NotBlank(message = "{validation.name.notblank}")
    private String nome;
}
```

```
spring.datasource.password=
spring.mail.password=
jwt.secret=string_de_pelo_menos_32_caracteresstring_de_pelo_menos_32_caracteres
jwt.expiration=1000000
```

```
spring.application.name=backend
spring.messages.basename=messages
spring.config.import=classpath:application-secrets.properties
```

```
spring.datasource.url=jdbc:mysql://localhost:35206/leilao
spring.datasource.username=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
# Criação da Tabela
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect;
```

```
#configuração email
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=frankwco@gmail.com
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

```
email=Email
#validações
validation.name.notblank=Nome obrigatório
validation.email.notblank={email} obrigatório
validation.email.notvalid={email} inválido
```

```
#regras de negócio  
pessoa.notfound = Pessoa não encontrada com o id {0}
```

```
#....
```

```
package com.leilao.backend.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PostMapping;  
import org.springframework.web.bind.annotation.RequestMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController  
@RequestMapping("/index")  
public class Index {  
  
    @GetMapping  
    public String index() {  
        return "Hello World Spring";  
    }  
  
    @GetMapping("/new")  
    public String index2() {  
        return "Hello World Spring 2";  
    }  
  
    @PostMapping  
    public String save() {  
        return "Success";  
    }  
  
}
```

```
package com.leilao.backend.controller;
```

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.messaging.handler.annotation.MessageMapping;  
import org.springframework.messaging.handler.annotation.SendTo;  
import org.springframework.stereotype.Controller;
```

```
import com.leilao.backend.model.Auction;  
import com.leilao.backend.model.AuctionBid;  
import com.leilao.backend.repository.AuctionBidRepository;  
import com.leilao.backend.repository.AuctionRepository;  
import com.leilao.backend.repository.PersonRepository;  
import com.leilao.backend.security.JwtService;
```

```
import lombok.AllArgsConstructor;
```

```

import lombok.Data;

@Controller
public class AuctionWebSocketController {

    @Autowired
    private AuctionBidRepository bidRepository;

    @Autowired
    private AuctionRepository auctionRepository;

    @Autowired
    private JwtService jwtService;

    @Autowired
    private PersonRepository personRepository;

    @RequestMapping("/bid/{auctionId}")
    @SendTo("/topic/auction/{auctionId}")
    public BidMessageReponse handleBid(BidMessageRequest bidMessage) {
        Auction auction = auctionRepository.findById(bidMessage.getAuctionId()).get();
        auction.setEmailUserBid(jwtService.extractUsername(bidMessage.getUserToken()));
        auction.setValueBid(auction.getValueBid()+auction.getIncrementValue());
        auctionRepository.save(auction);

        AuctionBid auctionBid = new AuctionBid();
        auctionBid.setAuction(auction);

        auctionBid.setPerson(personRepository.findByEmail(jwtService.extractUsername(bidMessage.getUserToken()))).get();
        bidRepository.save(auctionBid);

        return new BidMessageReponse(auction.getId(), auctionBid.getPerson().getEmail(),
        auction.getValueBid());
    }
}

@Data
class BidMessageRequest {
    private Long auctionId;
    private String userToken;
}

@Data
@AllArgsConstructor
class BidMessageReponse {
    private Long auctionId;
    private String emailUser;
    private Double newValue;
}

```

```
package com.leilao.backend.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.leilao.backend.model.Category;
import com.leilao.backend.service.CategoryService;

@RestController
@RequestMapping("/api/category")
@CrossOrigin
public class CategoryController {

    @Autowired
    private CategoryService categoryService;

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public Category create(@RequestBody Category category) {
        return categoryService.create(category);
    }

    @PutMapping
    @PreAuthorize("hasRole('ADMIN')")
    public Category update(@RequestBody Category category) {
        return categoryService.create(category);
    }

    @GetMapping
    public List<Category> listAll() {
        return categoryService.listAll();
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public void delete(@PathVariable("id") Long id) {
        categoryService.delete(id);
    }
}
```

```
}
```

```
REACT_APP_API_BASE_URL=http://localhost:8080/api/
```

```
{  
  "welcome": "Welcome",  
  "password": "Password",  
  "email": "E-mail",  
  "button": {  
    "login": "Login",  
    "save": "Save",  
    "remove": "Remove"  
  }  
}
```

```
import axios from 'axios';
```

```
const api = axios.create({  
  baseURL: process.env.REACT_APP_API_BASE_URL,  
  headers:{  
    'Content-Type':'application/json'  
  }  
});
```

```
api.interceptors.request.use(  
  config => {  
    const token = localStorage.getItem('token');  
    if (token) {  
      config.headers.Authorization = `Bearer ${token}`;  
    }  
    return config;  
  },  
  error => Promise.reject(error)  
);
```

```
/* api.interceptors.response.use(  
  response => response,  
  error => {  
    if (error.response && error.response.status === 403 && !error.config.url.includes("favicon.ico")  
&&  
    !error.config.url.includes(".css") &&  
    !error.config.url.includes(".js")) {  
      window.location.href = '/unauthorized';  
    }  
    return Promise.reject(error);  
  }  
); */
```

```
export default api;
```

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import LanguageDetector from 'i18next-browser-languagedetector';
import pt from '../locales/pt.json';
import en from '../locales/en.json';
```

```
const resources = {
  en: {
    translation: en,
  },
  pt: {
    translation: pt,
  },
};
```

```
i18n
  .use(LanguageDetector)
  .use(initReactI18next)
  .init({
    resources,
    fallbackLng: 'en',
    interpolation: {
      escapeValue: false,
    },
  });
```

```
export default i18n;
```

```
import React from "react";
import "../Footer.css";
```

```
const Footer = () =>{

  return(
    <div className="footer">
      <p>&copy;2024</p>
    </div>
  );
}
```

```
export default Footer;
```

```
import React from "react";
import { Menubar } from "primereact/menubar";
import { Button } from "primereact/button";
import { useNavigate } from "react-router-dom";
import "../Header.css";
```

```
const Header = () => {
  const navigate = useNavigate();
```

```

const items = [
  {
    label: "Home",
    icon: "pi pi-home",
    command: () => navigate("/"),
  },
  {
    label: "Categoria",
    icon: "pi pi-tags",
    command: () => navigate("/category"),
  },
  {
    label: "Leilão",
    icon: "pi pi-gavel",
    command: () => navigate("/auction"),
  },
];

const endTemplate = (
  <Button
    label="Sair"
    icon="pi pi-sign-out"
    className="p-button-text"
    onClick={() => {
      localStorage.removeItem("token");
      navigate("/login");
    }}
  />
);

return (
  <div className="header">
    <Menubar model={items} end={endTemplate} />
  </div>
);
};

```

```

export default Header;

```

```

import React from 'react';
import './Header.css';

```

```

//const Header = ({nome})=>{
const Header = (params) => {
  const {nome, idade} = params;
  return(
    <>

    <div className="header">

```

```

        <h1>Olá, {nome}</h1>
      </div>
    </>
  );
}
export default Header;

```

```
import React, { useState } from "react";
```

```
const Calculadora = () => {
```

```

  const [valor1, setValor1] = useState("");
  const [valor2, setValor2] = useState("");
  const [operador, setOperador] = useState("");
  const [resultado, setResultado] = useState("");

```

```

  const calcular = () => {
    const v1 = parseFloat(valor1);
    const v2 = parseFloat(valor2);
    if (isNaN(v1) || isNaN(v2)) {
      return setResultado('valores Inválidos');
    }
    switch (operador) {
      case '+':
        setResultado(v1 + v2);
        break;
      case '-':
        setResultado(v1 - v2);
        break;
      case '*':
        setResultado(v1 * v2);
        break;
      case '/':
        setResultado(v2 !== 0 ? v1 / v2 : 'Divisão por 0');
        break;
      default:
        setResultado('Operador Inválido!');
    }
  }
}

```

```

return (
  <>
    Valor 1:
    <input value={valor1} onChange={e => setValor1(e.target.value)} /><br />
    Valor 2:
    <input value={valor2} onChange={e => setValor2(e.target.value)} /><br />
    Operador:
    <input value={operador} onChange={e => setOperador(e.target.value)} /><br />
  </>
)

```

```

    <button onClick={() => calcular()}>Calcular</button><br /><br />
    {1 == 1 && resultado}
  </>
);
}

```

```
export default Calculadora;
```

```
import React, { useState, useEffect } from 'react';
```

```

const Home = () => {
  const [tarefas, setTarefas] = useState([]);

  useEffect(() => {
    let tarefas = JSON.parse(localStorage.getItem("tarefas")) || [];
    setTarefas(tarefas);
  }, []);

  return (
    <>
      <h1>Página Inicial</h1>
      {tarefas.map(tarefa => (
        <p key={tarefa.id}>{tarefa.descricao}</p>
      ))}
    </>
  );
}
export default Home;

```

```

import React, { useState, useEffect } from "react";
import { Button } from 'primereact/button';

```

```

const Cadastro = () => {

  const [tarefa, setTarefa] = useState({ id: 0, descricao: "", data: "" });
  const [tarefas, setTarefas] = useState([]);

  useEffect(() => {
    let tarefas = JSON.parse(localStorage.getItem("tarefas")) || [];
    setTarefa({ ...tarefa, id: tarefas.length });
  }, []);

  const cadastrar = () => {
    tarefas.push(tarefa);
    localStorage.setItem("tarefas", JSON.stringify(tarefas));
  }

  const atualizarValor = (event) => {

```

```

    setTarefa({ ...tarefa, [event.target.id]: event.target.value });
    console.log(tarefa);
  }

  return (
    <>
      <h1>Cadastro de Tarefas</h1>
      {tarefa.descricao}
      <input type="text" value={tarefa.descricao} id="descricao" onChange={atualizarValor} /><br />
      <input type="text" id="data" value={tarefa.data} onChange={atualizarValor} /><br /><br />
      <button onClick={cadastrar}>Cadastrar</button><br /><br />
      <Button label="Submit" />
    </>
  );
}
export default Cadastro;

```

```

import React, { useState, useEffect, useRef } from "react";
import { DataTable } from "primereact/datatable";
import { Column } from "primereact/column";
import { Button } from "primereact/button";
import { Dialog } from "primereact/dialog";
import { InputText } from "primereact/inputtext";
import { Toast } from "primereact/toast";
import { ConfirmDialog, confirmDialog } from "primereact/confirmdialog";
import CategoryService from "../services/CategoryService";

```

```

const Category = () => {
  const [categories, setCategories] = useState([]);
  const [category, setCategory] = useState({ name: "", observation: "" });
  const [dialogVisible, setDialogVisible] = useState(false);
  const [isEdit, setIsEdit] = useState(false);
  const [loading, setLoading] = useState(true);
  const toast = useRef(null);

```

```

  const categoryService = new CategoryService();

```

```

  useEffect(() => {
    loadCategories();
  }, []);

```

```

  const loadCategories = async () => {
    setLoading(true);
    try {
      const data = await categoryService.list();
      setCategories(data);
    } catch (error) {
      toast.current.show({

```

```

        severity: "error",
        summary: "Erro",
        detail: "Erro ao buscar as categorias!",
    });
} finally {
    setLoading(false);
}
};

const openNew = () => {
    setCategory({ name: "", observation: "" });
    setDialogVisible(true);
    setIsEdit(false);
};

const hideDialog = () => {
    setDialogVisible(false);
};

const saveCategory = async () => {
    try {
        if (isEdit) {
            await categoryService.update(category);
            toast.current.show({ severity: "success", summary: "Atualizado", detail: "Categoria atualizada com sucesso!" });
        } else {
            await categoryService.insert(category);
            toast.current.show({ severity: "success", summary: "Criado", detail: "Categoria criada com sucesso!" });
        }
        loadCategories();
    } catch (error) {
        toast.current.show({
            severity: "error",
            summary: "Erro",
            detail: "Erro ao salvar categoria",
        });
    } finally {
        hideDialog();
    }
};

const editCategory = (category) => {
    setCategory({ ...category });
    setDialogVisible(true);
    setIsEdit(true);
};

const confirmDeleteCategory = (category) => {
    confirmDialog({

```

```

        message: `Remover a categoria "${category.name}"?`,
        header: "Confirmação",
        icon: "pi pi-exclamation-triangle",
        accept: () => deleteCategory(category),
    });
};

const deleteCategory = async (category) => {
    try {
        await categoryService.delete(category.id);
        toast.current.show({ severity: "warn", summary: "Removido", detail: "Categoria removida com sucesso" });
        loadCategories();
    } catch (error) {
        toast.current.show({
            severity: "error",
            summary: "Erro",
            detail: "Erro ao remover a categoria",
        });
    }
};

const actionBodyTemplate = (rowData) => {
    return (
        <>
        <Button
            icon="pi pi-pencil"
            className="p-button-rounded p-button-success mr-2"
            onClick={() => editCategory(rowData)}
        />
        <Button
            icon="pi pi-trash"
            className="p-button-rounded p-button-danger"
            onClick={() => confirmDeleteCategory(rowData)}
        />
        </>
    );
};

const dialogFooter = (
    <div>
        <Button label="Cancelar" icon="pi pi-times" className="p-button-text"
onClick={hideDialog} />
        <Button label="Salvar" icon="pi pi-check" className="p-button-text"
onClick={saveCategory} />
    </div>
);

return (
    <div className="p-grid p-justify-center">

```

```

    <Toast ref={toast} />
    <ConfirmDialog acceptLabel="Sim" rejectLabel="Não"/>

    <Button label="Nova Categoria" icon="pi pi-plus" className="p-button-success"
onClick={openNew} />

    <DataTable
      value={categories}
      loading={loading}

    >
      <Column field="name" header="Nome"></Column>
      <Column field="observation" header="Observação"></Column>
      <Column body={actionBodyTemplate} header="Ações"></Column>
    </DataTable>

    <Dialog
      visible={dialogVisible}
      style={{ width: "30vw" }}
      header={isEdit ? "Editar Categoria" : "Nova Categoria"}
      modal
      footer={dialogFooter}
      onHide={hideDialog}
    >
      <div className="field">
        <label htmlFor="name">Nome</label>
        <InputText
          id="name"
          value={category.name}
          onChange={(e) => setCategory({ ...category, name: e.target.value })}
          required
        />
      </div>
      <div className="field">
        <label htmlFor="observation">Observação</label>
        <InputText
          id="observation"
          value={category.observation}
          onChange={(e) => setCategory({ ...category, observation: e.target.value })}
        />
      </div>
    </Dialog>
  </div>
);
};

export default Category;

import api from '../config/axiosConfig';

```

```
class BaseService{

  constructor(endPoint){
    this.api = api;
    this.endPoint = endPoint;
  }

  async insert(data){
    const response =
      await this.api.post(this.endPoint, data);
    return response.data;
  }

  async update(data){
    const response = await this.api.put(this.endPoint, data);
    return response.data;
  }

  async delete(id){
    const response = await
      this.api.delete(`${this.endPoint}/${id}`);
    return response.data;
  }

  async list(){
    const response = await this.api.get(this.endPoint);
    return response.data;
  }
}
export default BaseService;
```

```
import BaseService from "../BaseService";
```

```
class CategoryService extends BaseService{

  constructor(){
    super('category');
  }
}
export default CategoryService;
```

```
import BaseService from "../BaseService";
```

```
class PersonService extends BaseService{

  constructor(){
    super('person');
```

```

    }

    async login(credentials){
      const response = await
        this.api.post(`${this.endPoint}/login`, credentials);
      return response.data;
    }
  }

export default PersonService;

import BaseService from "../BaseService";

class AuctionService extends BaseService{

  constructor(){
    super('auction');
  }

  async listPublic(){
    const response = await
      this.api.get(`${this.endPoint}/public`);
    return response.data;
  }
}

export default AuctionService;

import React, { useState } from 'react';
import { Card } from 'primereact/card';
import { InputText } from 'primereact/inputtext';
import { Password } from 'primereact/password';
import { Button } from 'primereact/button';
import { useNavigate } from 'react-router-dom';
import { useTranslation } from 'react-i18next';
import './Login.css';
import PersonService from '../services/PersonService';

const Login = () => {
  const [user, setUser] = useState({ email: "", password: "" });
  const navigate = useNavigate();
  const { t } = useTranslation();
  const personService = new PersonService();

  const handleChange = (input) => {
    setUser({ ...user, [input.target.name]: input.target.value });
  }

  const login = async () => {
    //chamada para o back-end para verificar as credenciais

```

```

    try {
      const response = await personService.login(user);
      let token = response.token;
      localStorage.setItem("token", token);
      localStorage.setItem("email", user.email);
      navigate("/");
    } catch(err) {
      console.log(err);
      alert("usuário ou senha incorretos");
    }
  }

  return (
    <div className="flex justify-content-center align-items-center min-h-screen">
      <Card title="Login" className="p-4" style={{ width: '400px' }}>
        <h1 className='textColor'>Página de Login</h1>
        <div className="grid">
          <div className="field col-12">
            <label htmlFor="email">Email</label><br />
            <InputText onChange={handleChange} name="email" id="email"
className="w-full" />
          </div>
          <div className="field col-12">
            <label htmlFor="password">Senha</label><br />
            <Password onChange={handleChange} name="password" id="password"
feedback={false} toggleMask className="w-full" inputClassName="w-full" />
          </div>
        </div>
        <Button onClick={login} label={t('button.login')} className="w-full mt-3" />
      </Card>
    </div>
  );
}

export default Login;

import React from 'react';
import { Card } from 'primereact/card';
import { InputText } from 'primereact/inputtext';
import { Button } from 'primereact/button';

const RecuperarSenha = () => {
  return (
    <div className="flex justify-content-center align-items-center min-h-screen">
      <Card title="Recuperar Senha" className="p-4" style={{ width: '400px' }}>
        <div className="grid">
          <div className="field col-12">
            <label htmlFor="email">E-mail</label><br />
            <InputText id="email" className="w-full" placeholder="Digite seu e-mail" />
          </div>
        </div>
      </Card>
    </div>
  );
}

```

```

        </div>
        <Button label="Recuperar Senha" className="w-full mt-3" />
    </Card>
</div>
);
}

```

```
export default RecuperassrSenhas;
```

```

spring.application.name=backend
spring.messages.basename=messages
spring.config.import=classpath:application-secrets.properties

```

```

spring.datasource.url=jdbc:mysql://localhost:3304/leilao
spring.datasource.username=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
# Criação da Tabela
spring.jpa.hibernate.ddl-auto=update

```

```

#configuração email
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=frankwco@gmail.com
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true

```

```

spring.datasource.password=
spring.mail.password=
jwt.secret=string_de_pelo_menos_32_caracteres
jwt.expiration=1000000

```

```

email=Email
#validações
validation.name.notblank=Nome obrigatório
validation.email.notblank={email} obrigatório
validation.email.notvalid={email} inválido

```

```

#regras de negócio
pessoa.notfound = Pessoa não encontrada com o id {0}

```

```
#....
```

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

```

```

<modelVersion>4.0.0</modelVersion>
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.0</version>
    <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.leilao</groupId>
<artifactId>backend</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>backend</name>
<description>Projeto </description>
<url/>
<licenses>
    <license/>
</licenses>
<developers>
    <developer/>
</developers>
<scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
</scm>
<properties>
    <java.version>17</java.version>
</properties>
<dependencies>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.11.2</version>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.11.2</version>
        <scope>runtime</scope>
    </dependency>

    <dependency>

```

```
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.11.2</version>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-mail</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
```

```

        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

```

```

</dependencies>

```

```

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <configuration>
                <annotationProcessorPaths>
                    <path>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </path>
                </annotationProcessorPaths>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
    </plugins>
</build>

```

```

</project>

```

```

{
  "name": "front",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "@testing-library/dom": "^10.4.0",

```

```

"@testing-library/jest-dom": "^6.6.3",
"@testing-library/react": "^16.3.0",
"@testing-library/user-event": "^13.5.0",
"axios": "^1.11.0",
"primeicons": "^7.0.0",
"primereact": "^10.9.6",
"react": "^19.1.0",
"react-dom": "^19.1.0",
"react-router-dom": "^7.6.2",
"react-scripts": "5.0.1",
"web-vitals": "^2.1.4"
},
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
},
"eslintConfig": {
  "extends": [
    "react-app",
    "react-app/jest"
  ]
},
"browserslist": {
  "production": [
    ">0.2%",
    "not dead",
    "not op_mini all"
  ],
  "development": [
    "last 1 chrome version",
    "last 1 firefox version",
    "last 1 safari version"
  ]
}
}

```

REACT\_APP\_API\_BASE\_URL = <http://localhost:8080>

```
import BaseService from "../BaseService";
```

```

class AutenticacaoService extends BaseService{

  constructor(){
    super("/autenticacao");
  }
}

```

```
    async login(dados){
      const resposta = await this.api.post(`${this.endPoint}/login`, dados);
      return resposta;
    }
  }
}
export default AutenticacaoService;
```

```
import api from "../configs/axiosConfig";
```

```
class BaseService {
```

```
  constructor(endPoint) {
    this.endPoint = endPoint;
    this.api = api;
  }
```

```
  async inserir(dados){
    const resposta = await this.api.post(this.endPoint, dados);
    return resposta;
  }
```

```
  async alterar(dados) {
    const resposta = await this.api.put(this.endPoint, dados);
    return resposta;
  }
```

```
  async excluir(id){
    const resposta = await this.api.delete(`${this.endPoint}/${id}`);
    return resposta;
  }
```

```
  async buscarTodos(){
    console.log("AAAA")
    try {
      const resposta = await this.api.get(this.endPoint);
      return resposta;
    } catch (error) {
      console.log(error);
    }
  }
```

```
  }
}
export default BaseService;
```

```
import BaseService from "../BaseService";
```

```

class PerfilService extends BaseService{

  constructor(){
    super("/perfil");
  }
}
export default PerfilService;

import './App.css';
import Footer from './componesnts/footer/Footer';
import Header from './components/header/Header';
import Calculadora from './pages/calculadora/Calculadora';
import Home from './pages/home/Home';

import { BrowserRouter, Route, Routes } from 'react-router-dom';
import Cadastrso from './pages/tarefa/Cadastro';
import Login from './pages/login/Login';
import RotaPrivadaLayout from './components/layout/RotaPrivadaLayout';
import PadraoLayout from './components/layout/PadraoLayout';
import Perfil from './pages/perfil/Perfil';

function App() {
  return (
    <>
    {/* <Header nome="Frank" /> */}
    <BrowserRouter>
      <Routes>
        <Route element={<RotaPrivadaLayout/>}>
          <Route path="/" element={<PadraoLayout>
            <Home/>
          </PadraoLayout>} />
          <Route path="/perfil" element={<PadraoLayout>
            <Perfil />
          </PadraoLayout>} />
        </Route>

        <Route path="/calculadora" Component={Calculadora} />
        <Route path="/cadastro" Component={Cadastro} />
        <Route path="/login" Component={() => <Login />} />
      </Routes>
    </BrowserRouter>
    {/* <Footer /> */}
  </>
  );
}

export default App;

```

```
.footer {
  font-family: 'Lucida Sans', 'Lucida Sans Regular', 'Lucida Grande', 'Lucida Sans Unicode',
  Geneva, Verdana, sans-serif;
  background-color: #585555;
  height: 100px;
  border: 1px solid #2e2e2e;
  border-radius: 10px;
  padding: 20px;
  color: #fff;
}
```

```
.header{
  font-family: 'Lucida Sans', 'Lucida Sans Regular', 'Lucida Grande', 'Lucida Sans Unicode',
  Geneva, Verdana, sans-serif;
  background-color: #585555;
  height: 100px;
  border: 1px solid #2e2e2e;
  border-radius: 10px;
  padding: 20px;
  color: #fff;
}
```

```
import React from "react";
import Header from "../header/Header";
import Footer from "../footer/Footer";
```

```
const PadraoLayout = ({ children }) => {

  return (
    <>
      <Header />
      {children}
      <Footer />
    </>
  );
}
```

```
export default PadraoLayout;
```

```
import React, { useState, useEffect, useRef } from "react";
import { DataTable } from "primereact/datatable";
import { Column } from "primereact/column";
import { Button } from "primereact/button";
import { Dialog } from "primereact/dialog";
import { InputText } from "primereact/inputtext";
import { Toast } from "primereact/toast";
import { ConfirmDialog, confirmDialog } from "primereact/confirmdialog";
import PerfilService from "../../services/PerfilService";
```

```

const Perfil = () => {
  const [perfis, setPerfis] = useState([]);
  const [perfil, setPerfil] = useState({ nome: "" });
  const [dialogVisible, setDialogVisible] = useState(false);
  const [isEdit, setIsEdit] = useState(false);
  const [loading, setLoading] = useState(true);
  const toast = useRef(null);

  const perfilService = new PerfilService();

  useEffect(() => {
    carregarPerfis();
  }, []);

  const carregarPerfis = async () => {
    setLoading(true);
    try {
      const data = await perfilService.buscarTodos();
      console.log(data);
      setPerfis(data.data.content);
    } catch (error) {
      toast.current.show({
        severity: "error",
        summary: "Erro",
        detail: "Erro ao buscar os perfis!",
      });
    } finally {
      setLoading(false);
    }
  };

  const abrirNovo = () => {
    setPerfil({ nome: "" });
    setDialogVisible(true);
    setIsEdit(false);
  };

  const esconderDialog = () => {
    setDialogVisible(false);
  };

  const salvarPerfil = async () => {
    try {
      if (isEdit) {
        await perfilService.alterar(perfil);
        toast.current.show({ severity: "success", summary: "Atualizado", detail: "Perfil atualizado com sucesso!" });
      } else {
        await perfilService.inserir(perfil);
      }
    }
  };

```

```

        toast.current.show({ severity: "success", summary: "Criado", detail: "Perfil criado com
sucesso!" });
    }
    carregarPerfis();
} catch (error) {
    toast.current.show({
        severity: "error",
        summary: "Erro",
        detail: "Erro ao salvar perfil",
    });
} finally {
    esconderDialog();
}
};

```

```

const editarPerfil = (perfil) => {
    setPerfil({ ...perfil });
    setDialogVisible(true);
    setIsEdit(true);
};

```

```

const confirmarExclusaoPerfil = (perfil) => {
    confirmDialog({
        message: `Remover o perfil "${perfil.nome}"?`,
        header: "Confirmação",
        icon: "pi pi-exclamation-triangle",
        accept: () => excluirPerfil(perfil),
    });
};

```

```

const excluirPerfil = async (perfil) => {
    try {
        await perfilService.excluir(perfil.id);
        toast.current.show({ severity: "warn", summary: "Removido", detail: "Perfil removido com
sucesso" });
        carregarPerfis();
    } catch (error) {
        toast.current.show({
            severity: "error",
            summary: "Erro",
            detail: "Erro ao remover o perfil",
        });
    }
};

```

```

const actionBodyTemplate = (rowData) => {
    return (
        <>
        <Button
            icon="pi pi-pencil"

```

```

        className="p-button-rounded p-button-success mr-2"
        onClick={() => editarPerfil(rowData)}
      />
      <Button
        icon="pi pi-trash"
        className="p-button-rounded p-button-danger"
        onClick={() => confirmarExclusaoPerfil(rowData)}
      />
    </>
  );
};

const dialogFooter = (
  <div>
    <Button label="Cancelar" icon="pi pi-times" className="p-button-text"
onClick={esconderDialog} />
    <Button label="Salvar" icon="pi pi-check" className="p-button-text" onClick={salvarPerfil}
  />
  </div>
);

return (
  <div className="p-grid p-justify-center">
    <Toast ref={toast} />
    <ConfirmDialog acceptLabel="Sim" rejectLabel="Não" />

    <Button label="Novo Perfil" icon="pi pi-plus" className="p-button-success"
onClick={abrirNovo} />

    <DataTable
      value={perfis}
      loading={loading}
      paginator
      rows={10}
      rowsPerPageOptions={[5, 10, 25]}
    >
      <Column field="nome" header="Nome"></Column>
      <Column body={actionBodyTemplate} header="Ações"></Column>
    </DataTable>

    <Dialog
      visible={dialogVisible}
      style={{ width: "50vw" }}
      header={isEdit ? "Editar Perfil" : "Novo Perfil"}
      modal
      footer={dialogFooter}
      onHide={esconderDialog}
    >
      <div className="field">
        <label htmlFor="nome">Nome: </label>

```

```

        <InputText
          id="nome"
          value={perfil.nome}
          onChange={(e) => setPerfil({ ...perfil, nome: e.target.value })}
          required
          className="w-full"
        />
      </div>
    </Dialog>
  </div>
);
};

```

```
export default Perfil;
```

```

import React from "react";
import { Navigate, Outlet } from "react-router-dom";

```

```

const RotaPrivadaLayout = () =>{

  const usuario = localStorage.getItem("usuario")?true:false;

  return(
    usuario?<Outlet/>:<Navigate to="/login" replace/>
  );
}
export default RotaPrivadaLayout;

```

```
import axios from 'axios';
```

```

const api = axios.create({
  baseURL: process.env.REACT_APP_API_BASE_URL,
  headers: {
    'Content-Type': 'application/json'
  }
});

```

```

api.interceptors.request.use(
  config => {
    const usuario = JSON.parse(localStorage.getItem('usuario'));
    if (usuario) {
      config.headers.Authorization = `Bearer ${usuario.token}`;
    }
    return config;
  },
  error => Promise.reject(error)
);

```

```
export default api;
```

```
import React, { useState } from "react";
```

```
const Calculadora = () => {
```

```
  const [valor1, setValor1] = useState("");
  const [valor2, setValor2] = useState("");
  const [operador, setOperador] = useState("");
  const [resultado, setResultado] = useState("");
```

```
  const calcular = () => {
    const v1 = parseFloat(valor1);
    const v2 = parseFloat(valor2);
    if (isNaN(v1) || isNaN(v2)) {
      return setResultado('valores Inválidos');
    }
    switch (operador) {
      case '+':
        setResultado(v1 + v2);
        break;
      case '-':
        setResultado(v1 - v2);
        break;
      case '*':
        setResultado(v1 * v2);
        break;
      case '/':
        setResultado(v2 !== 0 ? v1 / v2 : 'Divisão por 0');
        break;
      default:
        setResultado('Operador Inválido!');
    }
  }
}
```

```
  return (
    <>
      Valor 1:
      <input value={valor1} onChange={e => setValor1(e.target.value)} /><br />
      Valor 2:
      <input value={valor2} onChange={e => setValor2(e.target.value)} /><br />
      Operador:
      <input value={operador} onChange={e => setOperador(e.target.value)} /><br />
      <button onClick={() => calcular()}>Calcular</button><br /><br />
      {1 == 1 && resultado}
    </>
  );
```

```
}
```

```
export default Calculadora;
```

```
import React, { useState, useEffect } from 'react';
```

```
const Home = () => {  
  const [tarefas, setTarefas] = useState([]);  
  
  useEffect(() => {  
    let tarefas = JSON.parse(localStorage.getItem("tarefas")) || [];  
    setTarefas(tarefas);  
  }, []);  
  
  return (  
    <>  
      <h1>Página Inicial</h1>  
      {tarefas.map(tarefa => (  
        <p key={tarefa.id}>{tarefa.descricao}</p>  
      ))}  
    </>  
  );  
}
```

```
export default Home;
```